

总结一下，算法竞赛的程序应当只做 3 件事情：读入数据、计算结果、打印输出。不要打印提示信息，不要在打印输出后“暂停程序”，更不要尝试画图、访问网络等与算法无关的任务。

回到刚才的程序，它多了几个新内容。首先是“`const double pi = acos(-1.0);`”。这里也声明了一个叫 `pi` 的“符号”，但是 `const` 关键字表明它的值是不可以改变的——`pi` 是一个真正的数学常数^①。

提示 1-7：尽量用 `const` 关键字声明常数。

接下来是 `s1 = pi * r * r`。这条语句应该如何理解呢？“`s1` 等于 `pi*r*r`”吗？并不是这样的。若把它换成“`pi * r * r = s1`”，编译器会给出错误信息：`invalid value in assignment`。如果这条语句真的是“二者相等”的意思，为何不允许反着写呢？

事实上，这条语句的学术说法是赋值（assignment），它不是一个描述，而是一个动作。其确切含义是：先把“等号”右边的值算出来，然后赋于左边的变量中。注意，变量是“喜新厌旧”的，即新的值将覆盖原来的值，一旦被赋了新的值，变量中原来的值就丢失了。

提示 1-8：赋值是个动作，先计算右边的值，再赋给左边的变量，覆盖它原来的值。

最后是“`Area = %.3fn`”，该语句的用法很容易被猜到：只有以“%”开头的部分才会被后面的值替换掉，其他部分原样输出。

提示 1-9：`printf` 的格式字符串中可以包含其他可打印符号，打印时原样输出。

这里还有一个非常容易忽略的细节：输入采用的是“`%lf`”而不是“`%f`”。关于这一点，本章的末尾会继续讨论，现在先跳过。

1.3 顺序结构程序设计

例题 1-2 三位数反转

输入一个三位数，分离出它的百位、十位和个位，反转后输出。

样例输入：

127

样例输出：

721

【分析】

首先将三位数读入变量 `n`，然后进行分离。百位等于 `n/100`（注意这里取的是商的整数部分），十位等于 `n/10%10`（这里的%是取余数操作），个位等于 `n%10`。程序如下：

^① 有的读者可能会用 `math.h` 中定义的常量 `M_PI`，但其实这个常数不是 ANSI C 标准的。不信可以用 `gcc-ansi` 编译试试。

程序 1-6 三位数反转 (1)

```
#include<stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    printf("%d%d%d\n", n%10, n/10%10, n/100);
    return 0;
}
```

此题有一个没有说清楚的细节，即：如果个位是 0，反转后应该输出吗？例如，输入是 520，输出是 025 还是 25？如果在算法竞赛中遇到这样的问题，可向监考人员询问^①。但是在这里，两种情况的处理方法都应学会。

提示 1-10：算法竞赛的题目应当是严密的，各种情况下的输出均应有严格规定。如果在比赛中发现题目有漏洞，应向相关人员询问，尽量不要自己随意假定。

上面的程序输出 025，但要改成输出 25 似乎会比较麻烦——必须判断 $n\%10$ 是不是 0，但目前还没有学到“根据不同情况执行不同指令”（分支结构程序设计是 1.4 节的主题）。

一个解决方法是在输出前把结果存储在变量 m 中。这样，直接用 $\%d$ 格式输出 m ，将输出 25。要输出 025 也很容易，把输出格式变为 $\%03d$ 即可。

程序 1-7 三位数反转 (2)

```
#include<stdio.h>
int main()
{
    int n, m;
    scanf("%d", &n);
    m = (n%10)*100 + (n/10%10)*10 + (n/100);
    printf("%03d\n", m);
    return 0;
}
```

例题 1-3 交换变量

输入两个整数 a 和 b ，交换二者的值，然后输出。

样例输入：

824 16

样例输出：

16 824

^① 如果是网络竞赛，还可以向组织者发信，在论坛中提问或者拨打热线电话。

【分析】

按照题目所说，先把输入存入变量 a 和 b ，然后交换。如何交换两个变量呢？最经典的方法是三变量法：

程序 1-8 变量交换（1）

```
#include<stdio.h>
int main()
{
    int a, b, t;
    scanf("%d%d", &a, &b);
    t = a;
    a = b;
    b = t;
    printf("%d %d\n", a, b);
    return 0;
}
```

可以将这种方法形象地比喻成将一瓶酱油和一瓶醋借助一个空瓶子进行交换：先把酱油倒入空瓶，然后将醋倒进原来的酱油瓶中，最后把酱油从辅助的瓶子中倒入原来的醋瓶子里。这样的比喻虽然形象，但是初学者应当注意它和真正的变量交换的区别。

借助一个空瓶子的目的是：避免把醋直接倒入酱油瓶子——直接倒进去，二者混合以后，将很难分开。在 C 语言中，如果直接进行赋值 $a=b$ ，则原来 a 的值（酱油）将会被新值（醋）覆盖，而不是混合在一起。

当酱油被倒入空瓶以后，原来的酱油瓶就变空了，这样才能装醋。但在 C 语言中，进行赋值 $t=a$ 后， a 的值不变，只是把值复制给了变量 t 而已，自身并不会变化。尽管 a 的值马上就会被改写，但是从原理上看， $t=a$ 的过程和“倒酱油”的过程有着本质区别。

提示 1-11：赋值 $a=b$ 之后，变量 a 原来的值被覆盖，而 b 的值不变。

另一个方法没有借助任何变量，但是较难理解：

程序 1-9 变量交换（2）

```
#include<stdio.h>
int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    a = a + b;
    b = a - b;
    a = a - b;
    printf("%d %d\n", a, b);
    return 0;
}
```